Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering and Research, Pune

Department of Computer Engineering

Subject: Operating System

Prepared By: Shrutika Menkudale
Asst. Prof. Computer Dept.

Introduction to Syllabus

Course :- PCC-203- COM: Operating Systems

Teaching /scheme	Credits	Examination Scheme
Theory : 03 Hours/Week	03	CCE : 30 Marks End-Semester: 70 Marks

Prerequisite Courses : Data Structure, Digital Electronics

Companion Course : Computer Organization and Microprocessors

Course Syllabus

Unit V - File Management and Disk Management (09 Hours)

File Management: File operation, Directory structure, File System structure, File Organization and Access, File Directories, File Allocation Methods,Secondary Storage Management, File Systems in Operating System (FAT, NTFS, EXT, and HFS).

Disk Management: Secondary-Storage Structure - Disk structure, Disk scheduling algorithm (FIFO, SSTF, SCAN, C-SCAN, LOOK, C-LOOK), Disk reliability, Disk formatting, Boot-block, Bad blocks.

Case Studies - 1) Study of Linux File System. 2) Study of Android File System.

Introduction

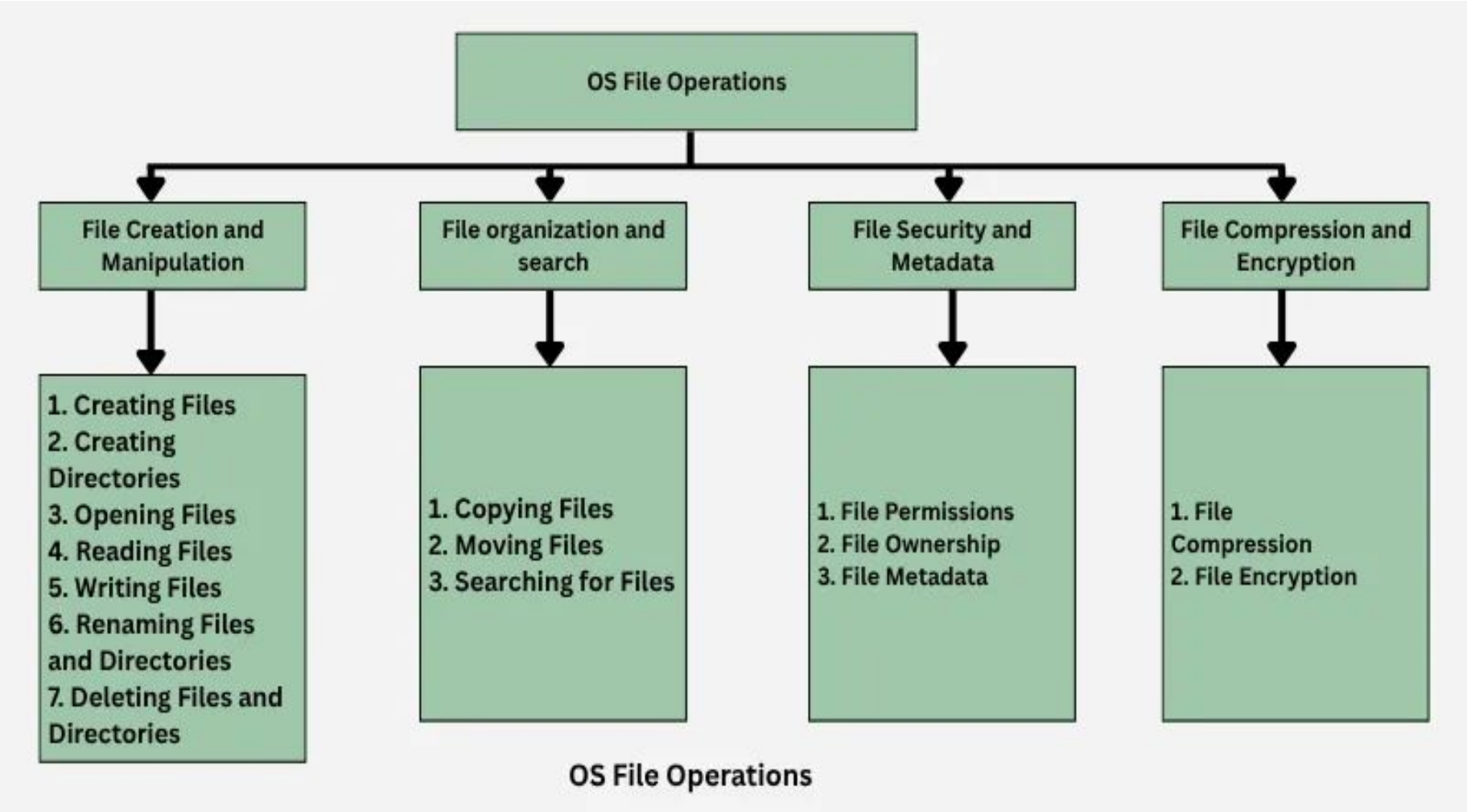
File management is the process of **organizing, storing, and retrieving digital files** for easy access and efficient use, involving methods like creating logical folder structures, implementing clear file naming conventions, and performing regular backups.

Introduction

File Operations

- File operations within an operating system (OS) **encompass a set of essential tasks and actions directed at files and directories residing within a computer's file system.**
- These operations are fundamental for the **effective management and manipulation** of data stored on various storage devices.

Introduction



File Creation and Manipulation

- File Creation and Manipulation encompasses essential operations within an operating system that involve **creating, modifying, and organizing files and directories.**
- These actions are vital for managing data efficiently and are integral to the functioning of computer systems.

File Management and Disk Management

File Operation	Description	System Calls / APIs
Creating Files	Create a new file for data storage.	• open() (Linux-like systems) • CreateFile() (Windows)
Creating Directories	Create a new directory for organizing files.	• mkdir() (Linux systems) • CreateDirectory() (Windows)
Opening Files	Open a file that you already have open to read or write from.	• open() (Linux systems) • CreateFile() (Windows)
Reading Files	Retrieve data from an open file.	• read() (Linux systems) • ReadFile() (Windows)

File Management and Disk Management

Writing Files	Store data in an open file.	• write() (Linux systems) • WriteFile() (Windows)
Renaming Files and Directories	If you want to rename a file or directory,.	• rename() (Linux systems) • MoveFile() (Windows)
Deleting Files and Directories	Remove files or directories.	• unlink() (Linux systems) • remove() (Linux systems) • DeleteFile() (Windows) • RemoveDirectory() (Windows)

File Organization and Search

File organization and search are key OS operations for arranging files systematically and swiftly locating specific data, optimizing file management and user efficiency.

File Operation	Description	System Calls / APIs
Copying Files	Create duplicates of files in another location.	• cp (Linux systems) • CopyFile() (Windows)
Moving Files	Relocate files from one location to another.	• mv (Linux systems) • MoveFile() (Windows)
Searching for Files	Locate files based on specific criteria.	• find (Linux systems) • FindFirstFile() and FindNextFile() (Windows)

File Security and Metadata

File Security and Metadata are vital components of file management, encompassing access control and crucial file information preservation within an operating system. They are essential for data security and efficient organization.

File Operation	Description	System Calls / APIs
File Permissions	Control access rights to files and directories.	• chmod (Linux systems) • SetFileSecurity (Windows)
File Ownership	Assign specific users or groups as file owners.	• chown (Linux systems) • SetFileSecurity (Windows)
File Metadata	Retrieve and manipulate file information.	• stat (Linux systems) • GetFileAttributesEx (Windows)

File Compression and Encryption

File Compression and Encryption are essential for optimizing storage and enhancing data security. Compression reduces file sizes, while encryption safeguards data privacy by making it unreadable without the correct decryption key.

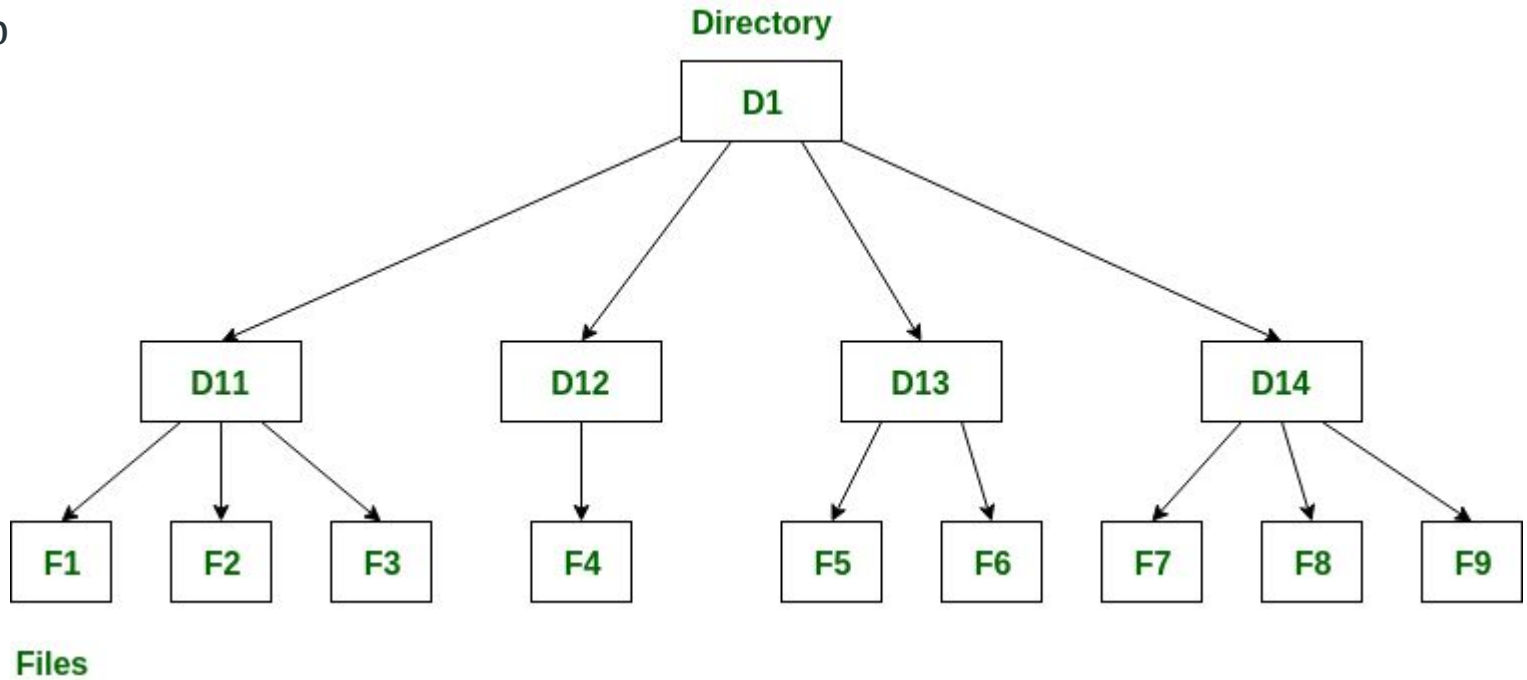
File Operation	Description	System Calls / APIs
File Compression	Reduce file sizes to save storage space.	• gzip, zip, tar (Linux systems), • Compress-Archive (Windows)
File Encryption	Protect data by converting it into an unreadable format.	• openssl, gpg (Linux systems) • Windows provides encryption libraries and APIs for encryption operations.

Structures of Directory in Operating System

The operating system uses directories to track where files are stored, just like using folders to organise papers.

- Different directory structures can be used to suit various organisational needs.
- Understanding directory structures helps in organising and accessing files more easily.

By using these structures, it becomes simpler to manage and navigate files on your comp



Different Types of Directories in OS

In an operating system, there are different types of directory structures that help organise and manage files efficiently. Each type of directory has its own way of arranging files and directories, offering unique benefits and features. These are

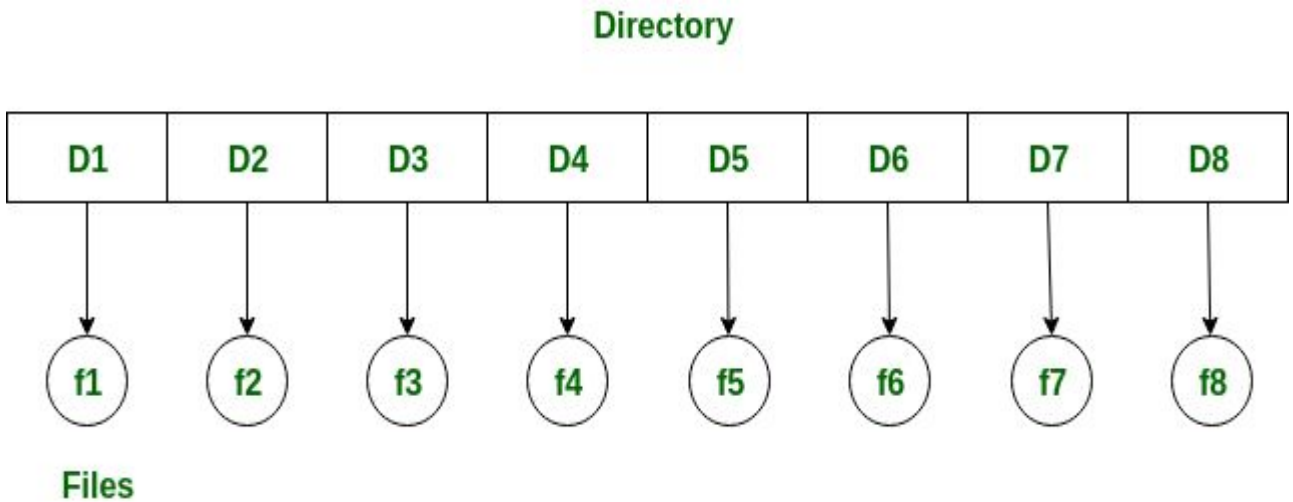
- Single-Level Directory
- Two-Level Directory
- Tree Structure/ Hierarchical Structure
- Acyclic Graph Structure
- General-Graph Directory Structure

Single-Level Directory

The single-level directory is the **simplest directory structure**. In it, all files are contained in the same directory which makes it easy to support and understand.

A single level directory has a significant limitation, however, when the number of files increases or when the system has more than one user. Since all the files are in the same directory, they must have a **unique name**. If two users call their dataset test, then the unique name rule violated.

- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.



Advantages

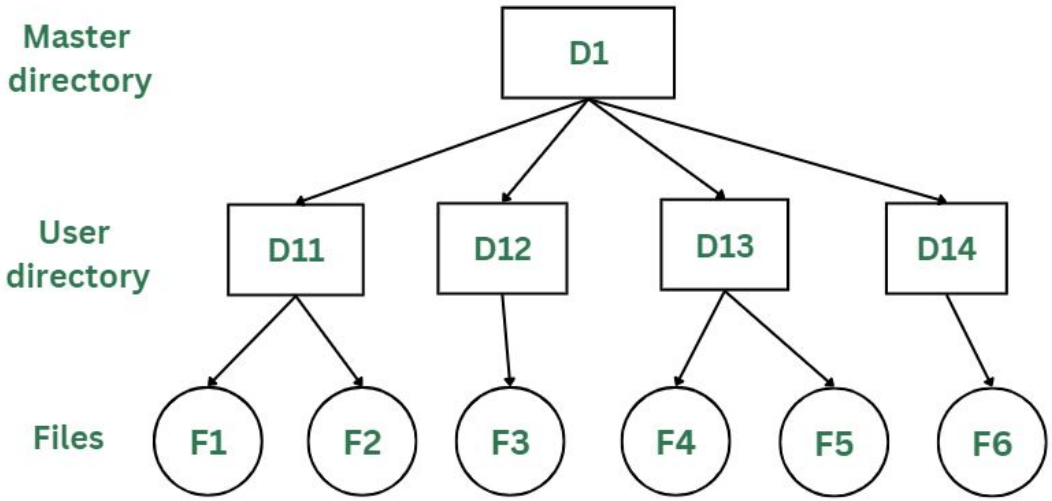
- **Logical Organization:** Arrange files hierarchically for easy navigation.
- **Efficiency:** Faster file searching and access.
- **Security:** Restrict access at directory level to protect data.
- **Backup & Recovery:** Simplifies locating and restoring files.
- **Scalability:** Easily supports growth with new files and directories

Disadvantages

- There may chance of name collision because two files can have the same name.
- Searching will become time taking if the directory is large.
- This can not group the same type of files together.

Two-Level Directory

In a two-level directory structure, each user has a separate User File Directory (UFD) containing only their files. A Master File Directory (MFD) stores entries for all users and points to their respective UFDs, preventing filename conflicts between users.



Advantages

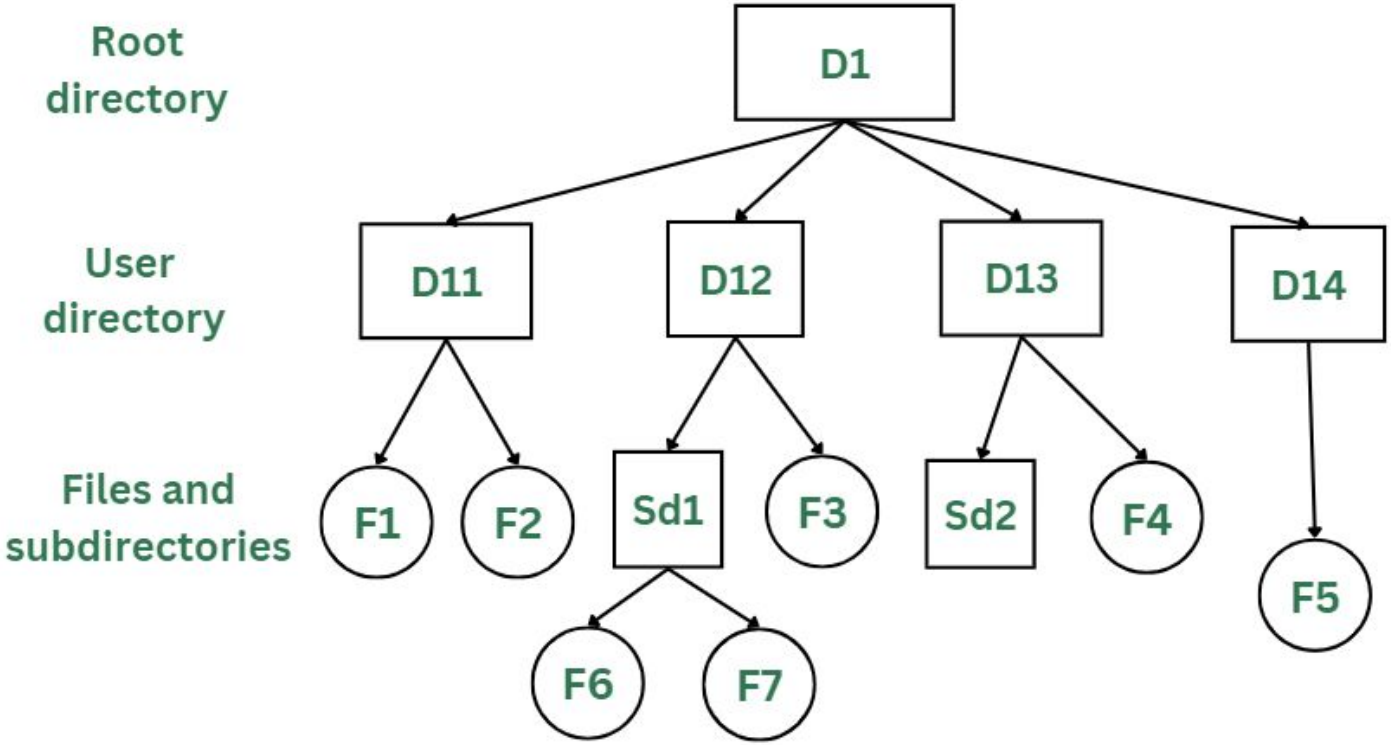
- The main advantage is there can be more than two files with same name, and would be very helpful if there are multiple users.
- A security would be there which would prevent user to access other user's files.
- Searching of the files becomes very easy in this directory structure.

Disadvantages

- As there is advantage of security, there is also disadvantage that the user cannot share the file with the other users.
- Unlike the advantage users can create their own files, users don't have the ability to create subdirectories.
- Scalability is not possible because one user can't group the same types of files together.

Tree Structure/ Hierarchical Structure

The tree directory structure is the most common in personal computers. It resembles an upside-down tree, with the root directory at the top containing all user directories. Each user can create files and subdirectories within their own directory but cannot access or modify the root or other users' directories.



Advantages

- This directory structure allows subdirectories inside a directory.
- The searching is easier.
- File sorting of important and unimportant becomes easier.
- This directory is more scalable than the other two directory structures explained.

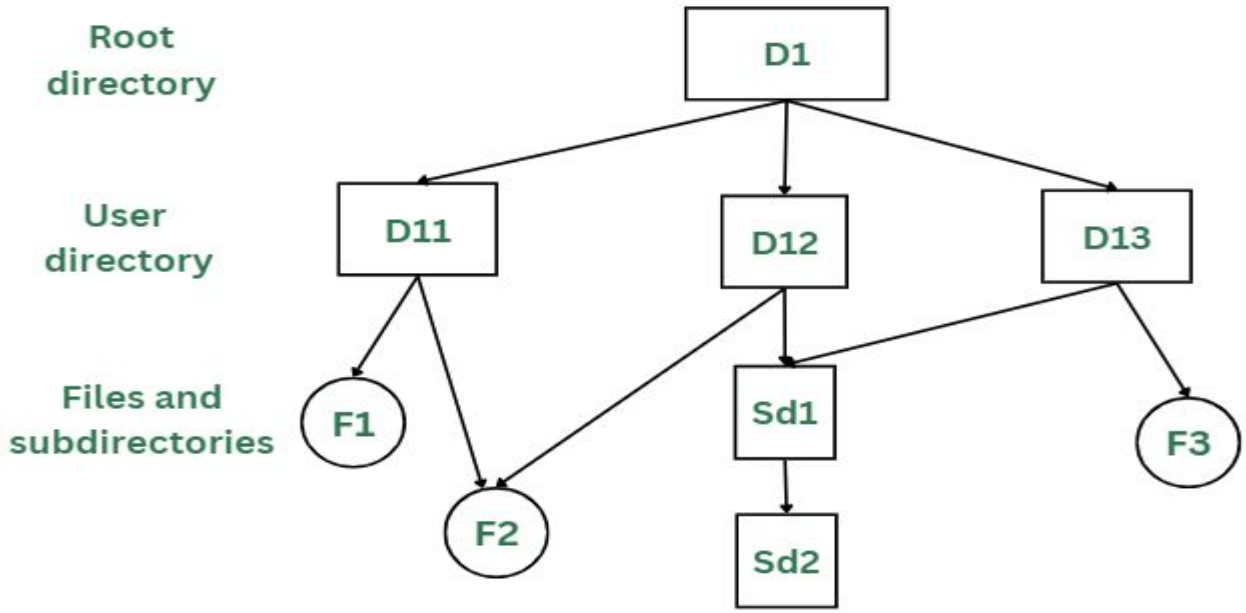
Disadvantages

- As the user isn't allowed to access other user's directory, this prevents the file sharing among users.
- As the user has the capability to make subdirectories, if the number of subdirectories increase the searching may become complicated.
- Users cannot modify the root directory data.
- If files do not fit in one, they might have to be fit into other directories.

Acyclic Graph Structure

In the earlier directory structures, a file could only be accessed from the directory it was stored in. The acyclic graph directory structure solves this by allowing a file or subdirectory to be shared across multiple directories using links. Changes made by one user are visible to all users sharing that file.

In the below figure, this explanation can be nicely observed, where a file is shared between multiple users. If any user makes a change, it would be reflected to both the users.



Advantages

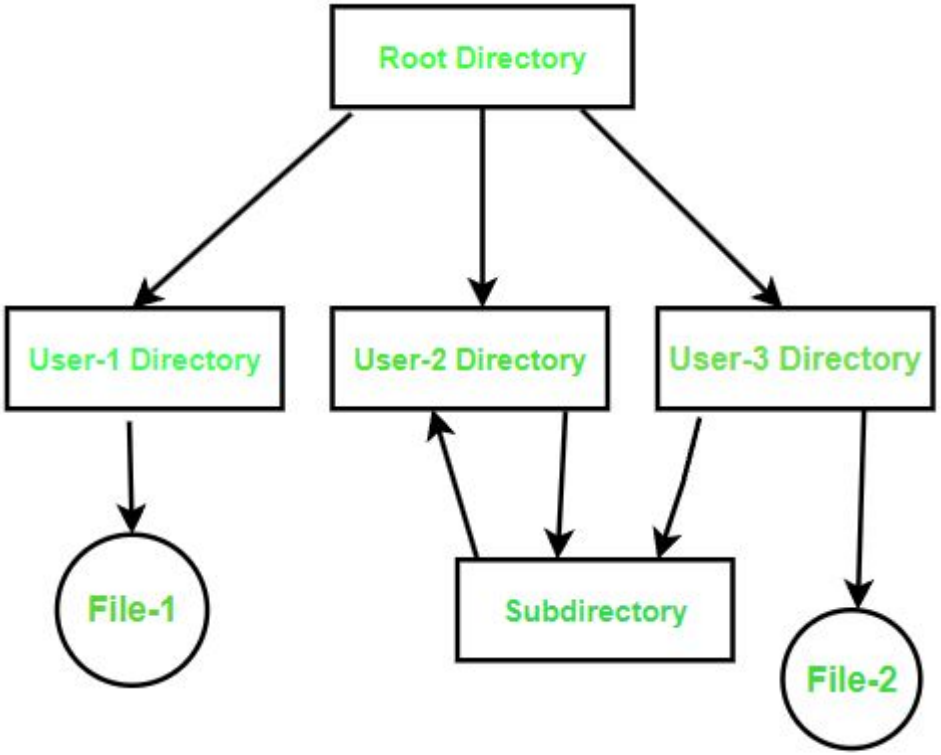
- Sharing of files and directories is allowed between multiple users.
- Searching becomes too easy.
- Flexibility is increased as file sharing and editing access is there for multiple users.

Disadvantages

- Because of the complex structure it has, it is difficult to implement this directory structure.
- The user must be very cautious to edit or even deletion of file as the file is accessed by multiple users.
- If we need to delete the file, then we need to delete all the references of the file inorder to delete it permanently.

General-Graph Directory Structure

Unlike the acyclic-graph directory, which avoids loops, the general-graph directory can have cycles, meaning a directory can contain paths that loop back to the starting point. This can make navigating and managing files more complex.



Advantages of General-Graph Directory

- More flexible than other directory structures.
- Allows cycles, meaning directories can loop back to each other.

Disadvantages of General-Graph Directory

- More expensive to implement compared to other solutions.
- Requires garbage collection to manage and clean up unused files and directories.

File Allocation Methods

The allocation methods define how the files are stored in the disk blocks. There are three main disk space or file allocation methods.

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

Contiguous Allocation

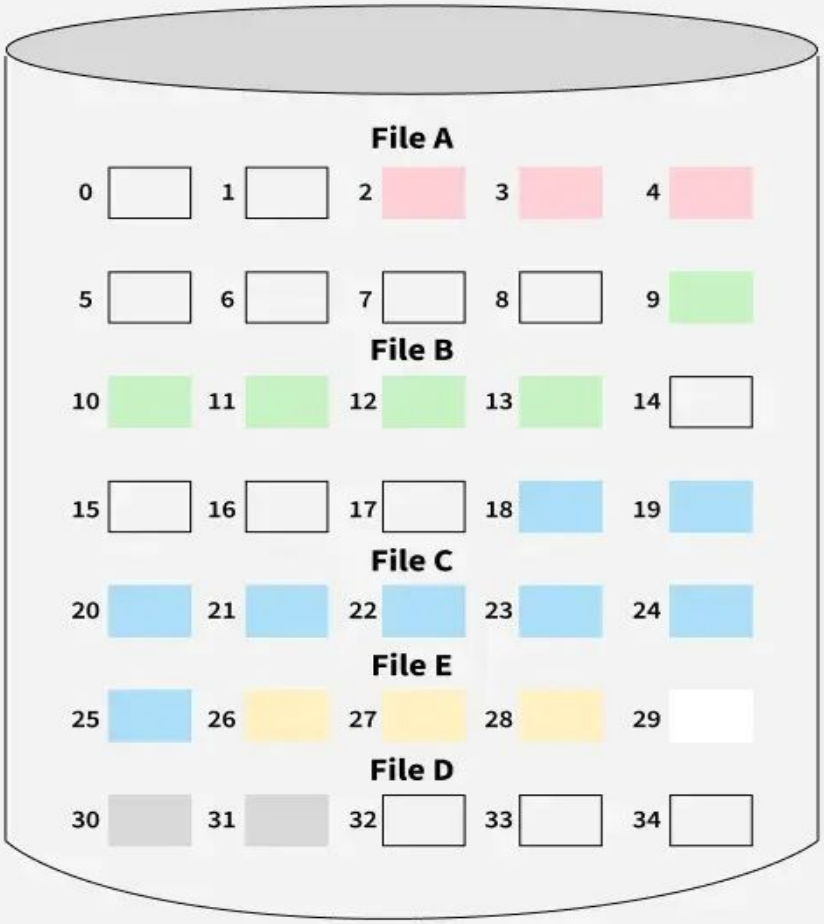
In this scheme, each file occupies a contiguous set of blocks on the disk. This means that given the starting block address and the length of the file (in terms of blocks required), we can determine the blocks occupied by the file. The directory entry for a file with contiguous allocation contains:

- Address of starting block
- Length of the allocated portion.

Example: If a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be:

b, b+1, b+2,.....b+n-1.

Continuous Allocation



File Name	Star block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Advantages

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the kth block of the file which starts at block b can easily be obtained as (b+k)
- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

Disadvantages

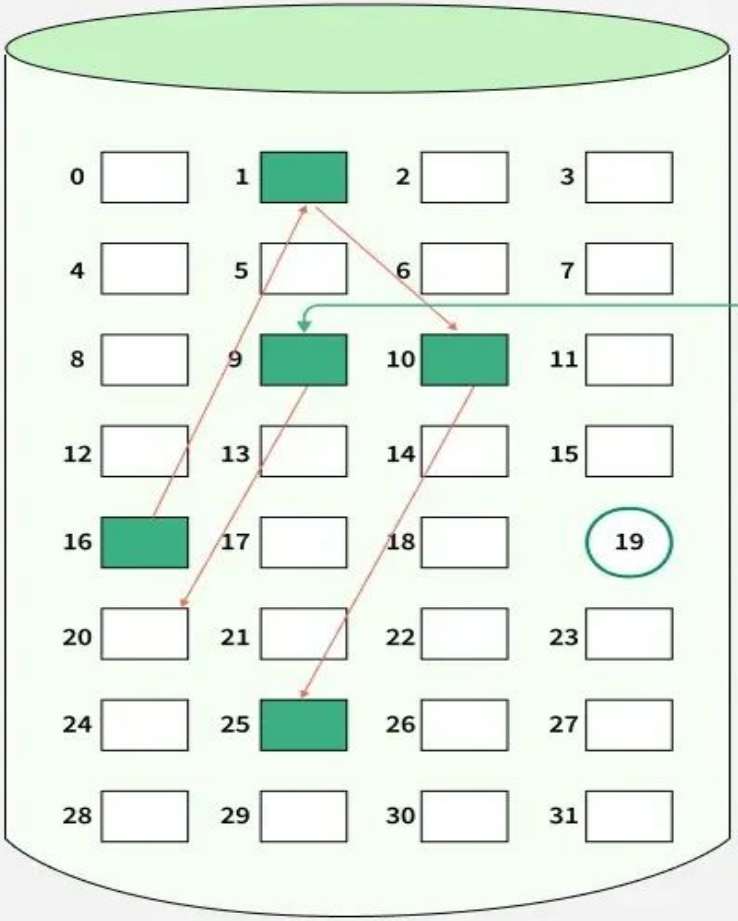
- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

Linked Allocation

In this scheme, each file is a linked list of disk blocks which need not be contiguous. Here,

- The disk blocks can be scattered anywhere on the disk. The directory entry contains a pointer to the starting and the ending file block.
- Each block contains a pointer to the next block occupied by the file.
- **Example:** The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.

Linked Allocation



Directory

File	start	end
jeep	9	25

Advantages

- This is very flexible in terms of file size. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method does not suffer from external fragmentation. This makes it relatively better in terms of memory utilization.

Disadvantages

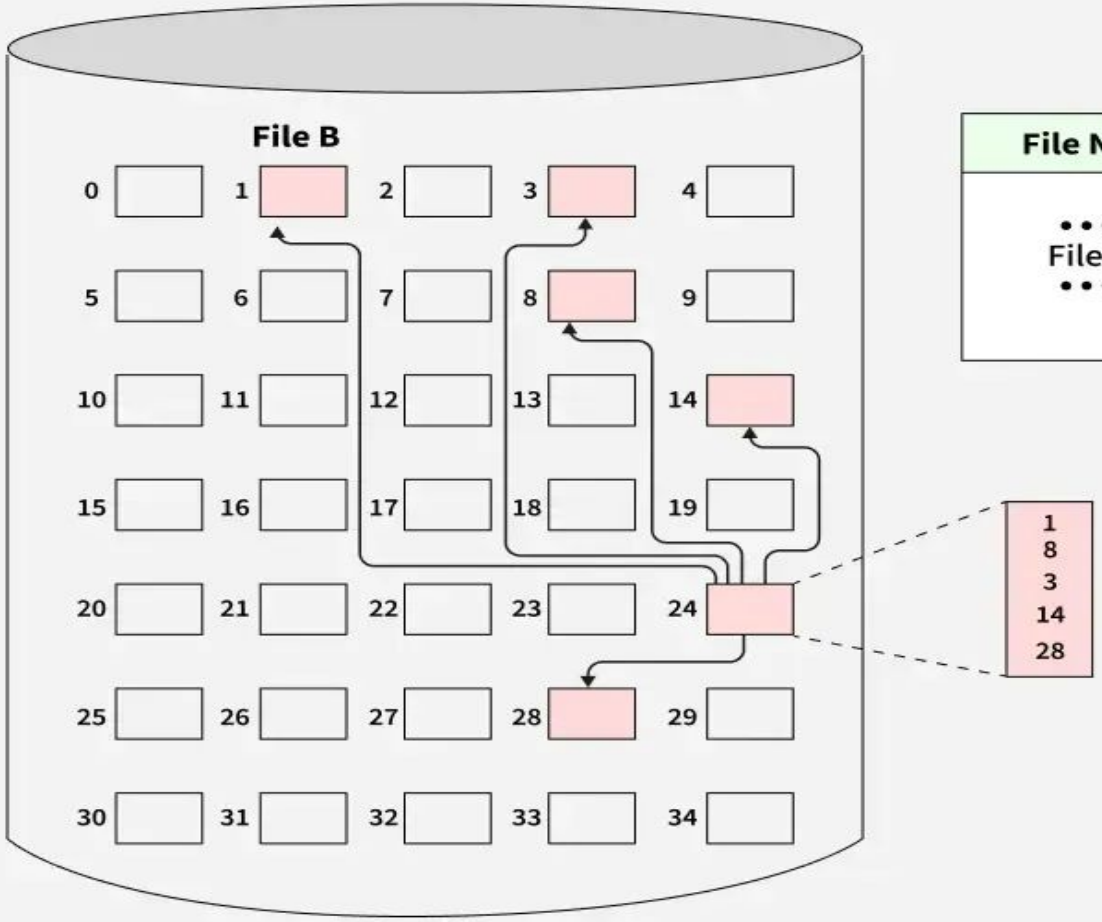
- Because the file blocks are distributed randomly on the disk, a large number of seeks are needed to access every block individually. This makes linked allocation slower.
- It does not support random or direct access. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.
- Pointers required in the linked allocation incur some extra overhead.

Indexed Allocation

In this scheme, a special block known as the Index block contains the pointers to all the blocks occupied by a file. Here,

- Each file has its own index block.
- The ith entry in the index block contains the disk address of the ith file block.
- The directory entry contains the address of the index block as shown in the image.

Indexed Allocation



File Allocation Table

File Name	Index block
...	...
File B	24
...	...

Advantages

- This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- It overcomes the problem of external fragmentation.

Disadvantages

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

Secondary Memory

Primary memory has limited storage capacity and is volatile. Secondary memory overcomes this limitation by providing permanent storage of data and in bulk quantity. **Secondary memory is also termed external memory** and refers to the various storage media on which a computer can store data and programs. The Secondary storage media can be fixed or removable.

Secondary memory is a type of computer memory that is **used for long-term storage of data and programs.** It is also known as auxiliary memory or external memory, and is distinct from primary memory, which is used for short-term storage of data and instructions that are currently being processed by the CPU.

Primary Memory	Secondary Memory
Primary memory is **directly accessed by the Central Processing Unit(CPU).**	Secondary memory is not accessed directly by the Central Processing Unit(CPU). **Instead, data accessed from a secondary memory is first loaded into Random Access Memory(RAM) and is then sent to the Processing Unit.**
RAM provides a **much faster-accessing speed to data than secondary memory**. By loading software programs and required files into primary memory(RAM), computers can process data much more quickly.	Secondary Memory **is slower in data accessing.** Typically primary memory is six times faster than secondary memory.
Primary memory, i.e. Random Access Memory(RAM) is volatile and gets completely erased when a computer is shut down.	Secondary **memory provides a feature of being non-volatile, which means it can hold on to its data** with or without electrical power supply.

Uses of Secondary Media:

- **Permanent Storage:** Primary Memory (RAM) is volatile, i.e. it loses all information when the **electricity is turned off, so in order to secure the data permanently in the device,** Secondary storage devices are needed.
- **Portability:** Storage mediums, like CDs, flash drives can be used to transfer the data from one device to another.

Fixed and Removable Storage

Fixed Storage-

Fixed storage is an internal media device that is used by a computer system to store data, and usually, these are referred to as the Fixed disk drives or Hard Drives.

Fixed **storage devices are literally not fixed, obviously, these can be removed from the system for repairing work, maintenance purposes, and also for an upgrade, et**c. But in general, this can't be done without a proper toolkit to open up the computer system to provide physical access, and that needs to be done by an engineer.

Types of fixed storage:

Internal flash memory (rare)

SSD (solid-state disk) units

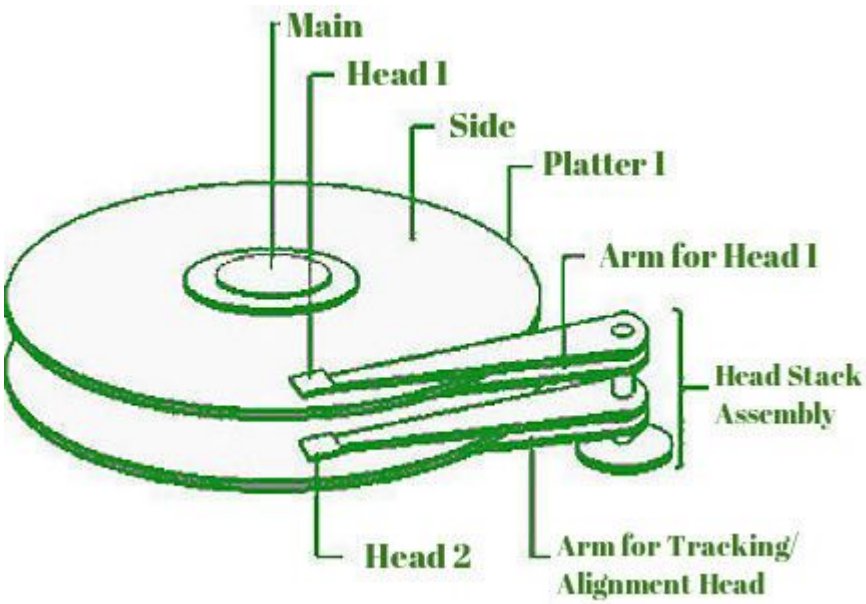
Hard disk drives (HDD)

Types of Removable Storage:

- Optical discs (CDs, DVDs, Blu-ray discs)
- Memory cards
- Floppy disks
- Magnetic tapes
- Disk packs
- Paper storage (punched tapes, punched cards)

1. Magnetic storage media: Magnetic media is coated with a magnetic layer that is magnetized in clockwise or anticlockwise directions. When the disk moves, the head interprets the data stored at a specific location in binary 1s and 0s at reading.

Examples: hard disks, floppy disks, and magnetic tapes.



2. Optical storage media

In optical storage, media information is stored and read using a laser beam. The data is stored as a spiral pattern of pits and ridges denoting binary 0 and binary 1.

Examples: CDs and DVDs

CD- ROM	CD-R	CD-RW
It stands for Compact Disk – Read Only Memory	It stands for Compact Disk-Recordable.	It stands for Compact Disk-Rewritable.
Data is written on these disks at the time of manufacture. This data cannot be changed, once is it written by the manufacturer, but can only be read. CD- ROMs are used for text, audio and video distribution like games, encyclopedias, and application software.	Data can be recorded on these disks but only once. Once the data is written in a CD-R, it cannot be erased/modified.	It can be read or written multiple times but a CD-RW drive needs to be installed on your computer before editing a CD-RW.

Advantages:

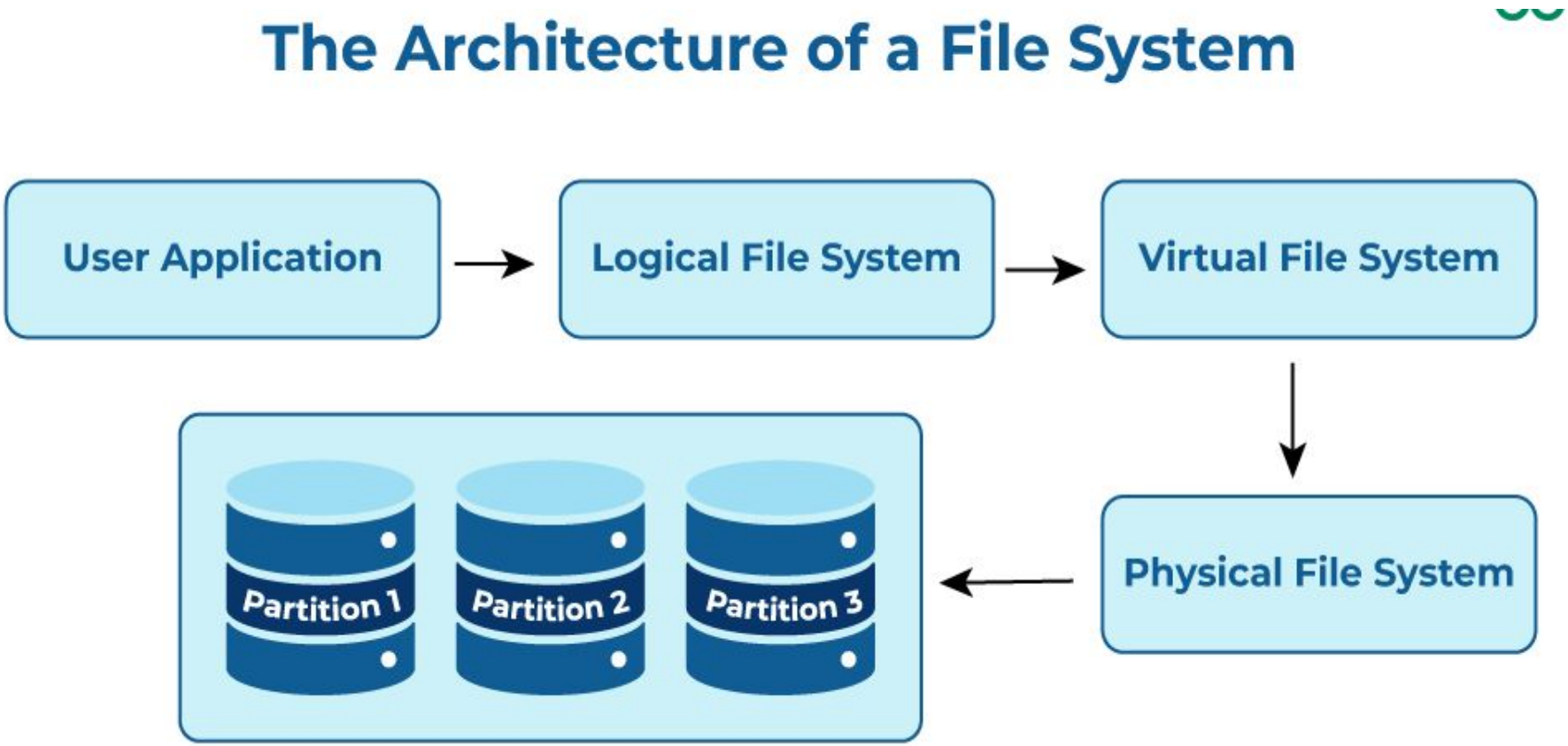
1. Large storage capacity: Secondary memory devices typically have a much larger storage capacity than primary memory, allowing users to store large amounts of data and programs.
2. Non-volatile storage: Data stored on secondary memory devices is typically non-volatile, meaning it can be retained even when the computer is turned off.
3. Portability: Many secondary memory devices are portable, making it easy to transfer data between computers or devices.
4. Cost-effective: Secondary memory devices are generally more cost-effective than primary memory.

Disadvantages:

1. Slower access times: Accessing data from secondary memory devices typically takes longer than accessing data from primary memory.
2. Mechanical failures: Some types of secondary memory devices, such as hard disk drives, are prone to mechanical failures that can result in data loss.
3. Limited lifespan: Secondary memory devices have a limited lifespan, and can only withstand a certain number of read and write cycles before they fail.
4. Data corruption: Data stored on secondary memory devices can become corrupted due to factors such as electromagnetic interference, viruses, or physical damage.
5. Overall, secondary memory is an essential component of modern computing systems, but it also has its limitations and drawbacks. The choice of a particular secondary memory device depends on the user's specific needs and requirements.

File Systems in Operating System

File systems are a crucial part of any operating system, providing a structured way to store, organize and manage data on storage devices such as hard drives, SSDs and USB drives.



- **User Application:** The programs or software that request file operations like read, write, or delete.
- **Logical File System:** Manages metadata, file names, directories, and access permissions.
- **Virtual File System (VFS):** Acts as a bridge, allowing different file systems to work under a single interface.
- **Physical File System:** Handles the actual storage of data blocks on the disk.
- **Partition 1, Partition 2, Partition 3:** Divisions of the storage device where files are stored physically.

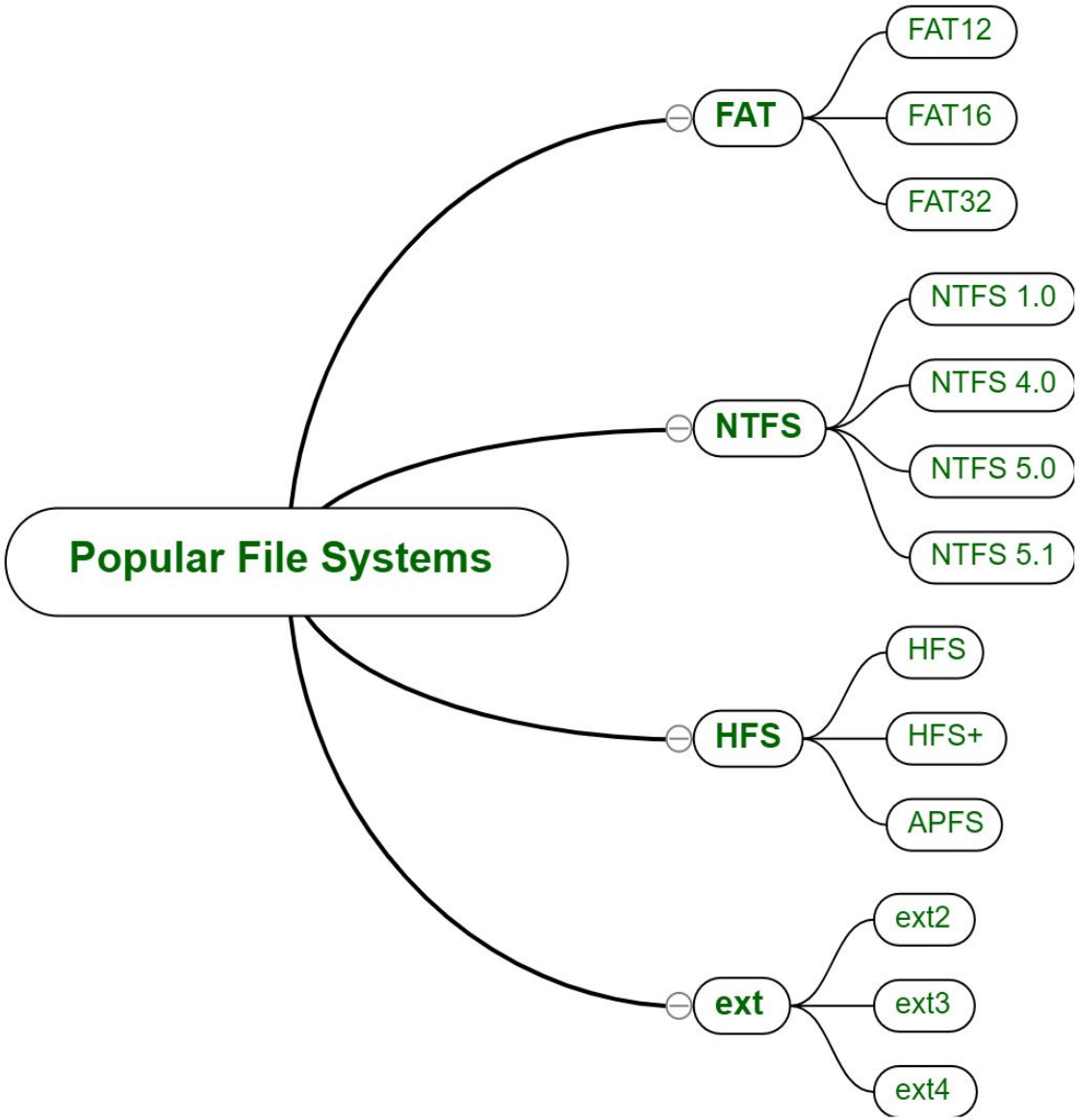
Popular File Systems

Some common types of file systems include:

- **FAT (File Allocation Table):** An older file system used by older versions of Windows and other operating systems.
- **NTFS (New Technology File System):** A modern file system used by Windows. It supports features such as file and folder permissions, compression and encryption.
- **ext (Extended File System):** A file system commonly used on Linux and Unix-based operating systems.
- **HFS (Hierarchical File System):** A file system used by macOS.
- **APFS (Apple File System):** A new file system introduced by Apple for their Macs and iOS devices.

The name of the file is divided into two parts as shown below:

1. Name
2. Extension, separated by a period.



Issues Handled By File System

- A free space is created on the hard drive whenever a file is deleted from it.
- To reallocate them to other files, many of these spaces may need to be recovered.
- Choosing where to store the files on the hard disc is the main issue with files one block may or may not be used to store a file.

File Types and Their Content

File Type	Usual extension	Function
Executable	exe, com, bin	Read to run machine language program
Object	obj, o	Compiled, machine language not linked
Source Code	C, java, pas, asm, a	Source code in various languages
Batch	bat, sh	Commands to the command interpreter
Text	txt, doc	Textual data, documents
Word Processor	wp, tex, rrf, doc	Various word processor formats
Archive	arc, zip, tar	Related files grouped into one compressed file
Multimedia	mpeg, mov, rm	For containing audio/video information
Markup	xml, html, tex	It is the textual data and documents
Library	lib, a, so, dll	It contains libraries of routines for programmers
Print or View	gif, pdf, jpg	It is a format for printing or viewing an ASCII or binary file.

Disk Scheduling Algorithms

Disk scheduling algorithms manage how data is read from and written to a computer's hard disk. These algorithms help determine the order in which disk read and write requests are processed.

- Disk scheduling is also known as I/O Scheduling.
- The main goals of disk scheduling are to optimize the performance of disk operations, reduce the time it takes to access data and improve overall system efficiency.
- Common disk scheduling methods include First-Come, First-Served (FCFS), Shortest Seek Time First (SSTF), SCAN, C-SCAN, LOOK, and C-LOOK.

Importance of Disk Scheduling in Operating System

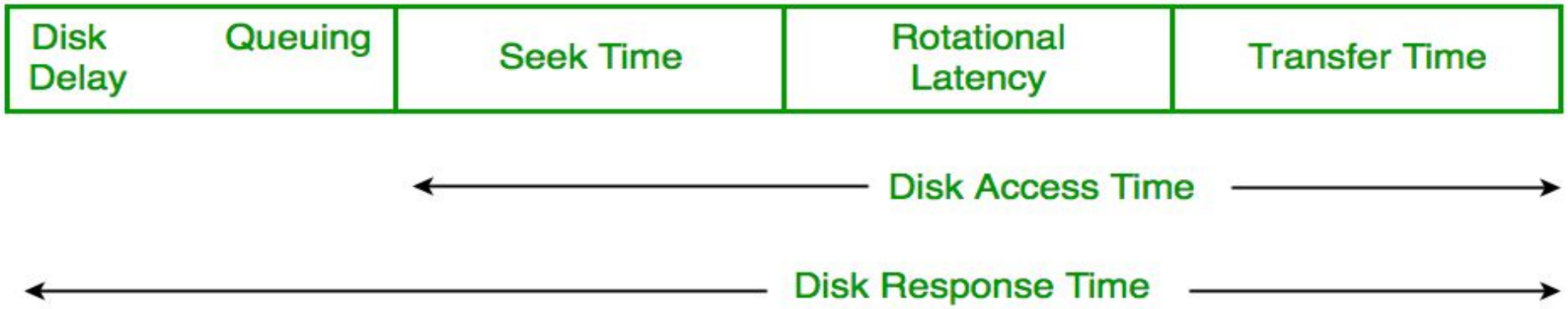
- Multiple I/O requests may arrive by different processes and only one I/O request can be served at a time by the disk controller. Thus other I/O requests need to wait in the waiting queue and need to be scheduled.
- Two or more requests may be far from each other so this can result in greater disk arm movement.
- Hard drives are one of the slowest parts of the computer system and thus need to be accessed in an efficient manner.

Key Terms Associated with Disk Scheduling

- **Seek Time**: Time taken to move the disk arm to the track where data is located.
- **Rotational Latency**: Time taken for the desired sector to rotate under the read/write head.
- **Transfer Time**: Time taken to actually read or write the data, depending on disk speed and data size.

Disk Access Time = Seek Time + Rotational Latency + Transfer Time

*Total Seek Time = Total head Movement * Seek Time*



Disk Response Time

- **Response Time**: The time a request waits before its I/O operation starts.
- **Average Response Time**: The mean waiting time of all requests.
- **Variance in Response Time**: How much individual waiting times differ from the average.

Goals of Disk Scheduling Algorithms

- Minimize Seek Time
- Maximize Throughput
- Minimize Latency
- Ensuring Fairness
- Efficiency in Resource Utilization

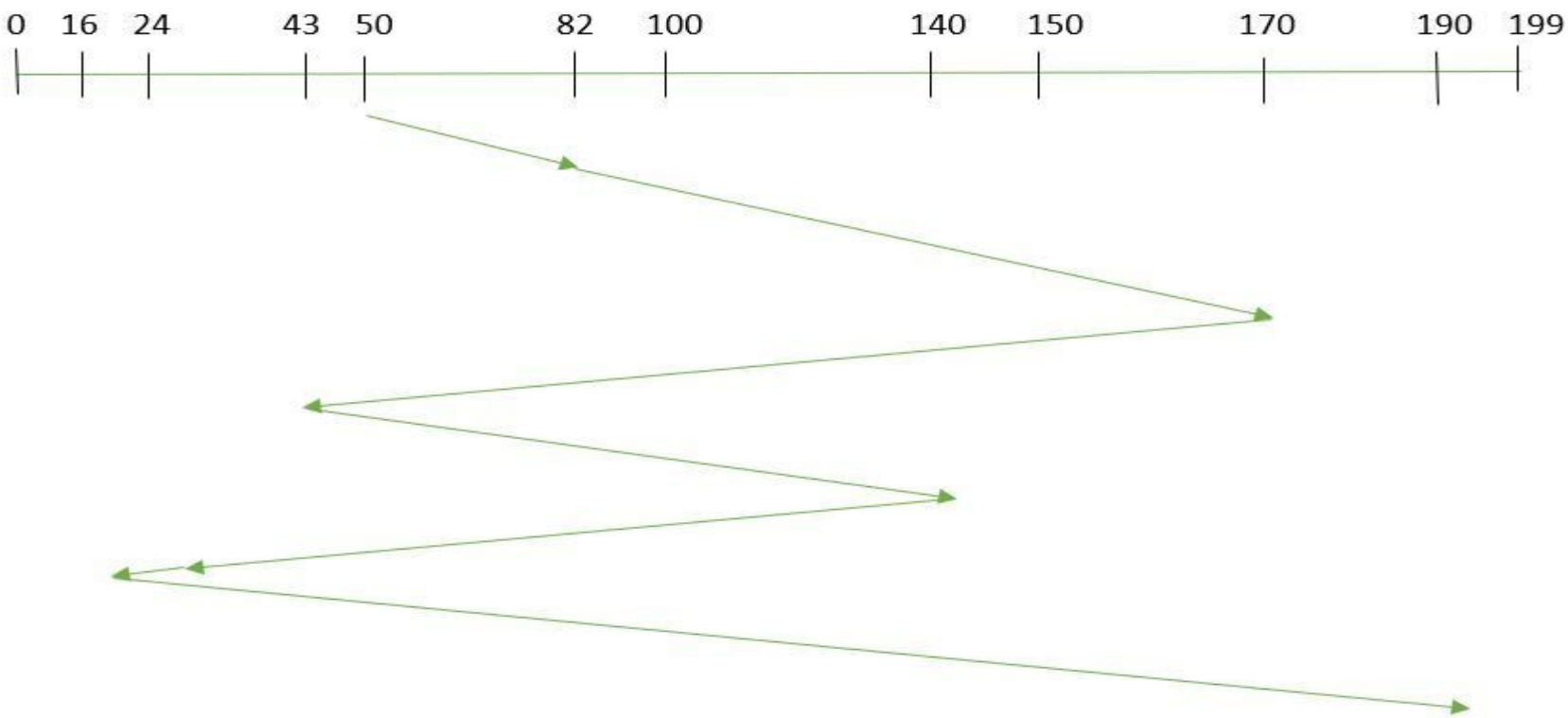
Disk Scheduling Algorithms

There are several Disk Several Algorithms. We will discuss in detail each one of them.

- **FCFS (First Come First Serve)**
- **SSTF (Shortest Seek Time First)**
- **SCAN**
- **C-SCAN**
- **LOOK**
- **C-LOOK**
- RSS (Random Scheduling)
- LIFO (Last-In First-Out)
- N-STEP SCAN
- F-SCAN

1. FCFS (First Come First Serve)

FCFS is the simplest of all Disk Scheduling Algorithms. In FCFS, the requests are addressed in the order they arrive in the disk queue. Let us understand this with the help of an example.



Suppose the order of request is- (82,170,43,140,24,16,190) and current position of Read/Write head is: 50

So, total overhead movement (total distance covered by the disk arm) =

(82-50)+(170-82)+(170-43)+(140-43)+(140-24)+(24-16)+(190-16) =642

Advantages of FCFS

Here are some of the advantages of First Come First Serve.

- Every request gets a fair chance
- No indefinite postponement

Disadvantages of FCFS

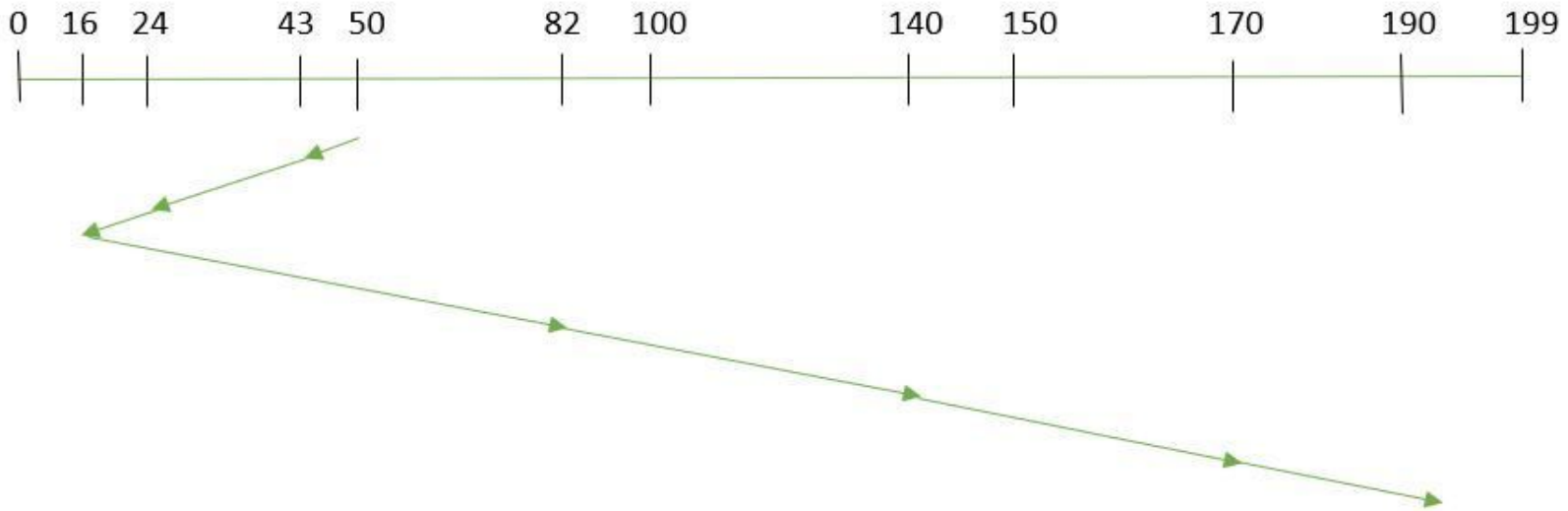
Here are some of the disadvantages of First Come First Serve.

- Does not try to optimize seek time
- May not provide the best possible service

2. SSTF (Shortest Seek Time First)

In SSTF (Shortest Seek Time First), requests having the shortest seek time are executed first. So, the seek time of every request is calculated in advance in the queue and then they are scheduled according to their calculated seek time. As a result, the request near the disk arm will get executed first. SSTF is certainly an improvement over FCFS as it decreases the average response time and increases the throughput of the system. Let us understand this with the help of an example.

Example:



Suppose the order of request is-
(82,170,43,140,24,16,190) and current position of Read/Write head is: 50

total overhead movement (total distance covered by the disk arm) =
(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) =208

Advantages of Shortest Seek Time First

Here are some of the advantages of Shortest Seek Time First.

- The average Response Time decreases
- Throughput increases

Disadvantages of Shortest Seek Time First

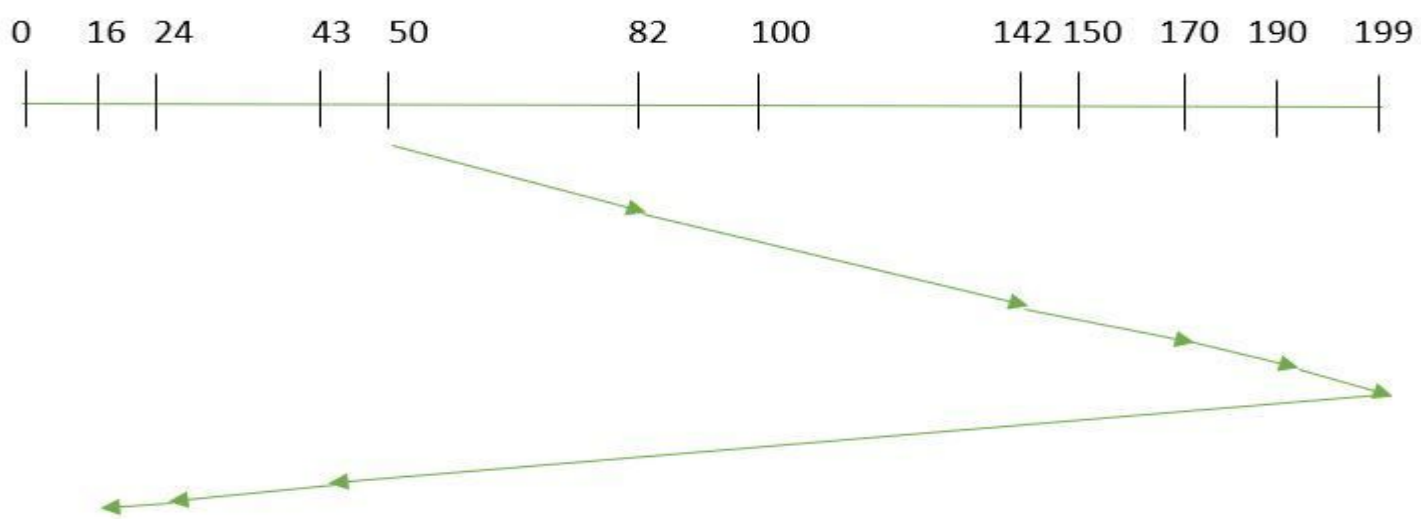
Here are some of the disadvantages of Shortest Seek Time First.

- Overhead to calculate seek time in advance
- Can cause Starvation for a request if it has a higher seek time as compared to incoming requests
- The high variance of response time as SSTF favors only some requests

3. SCAN

In the SCAN algorithm the disk arm moves in a particular direction and services the requests coming in its path and after reaching the end of the disk, it reverses its direction and again services the request arriving in its path. So, this algorithm works as an elevator and is hence also known as an **elevator algorithm.** As a result, the requests at the midrange are serviced more and those arriving behind the disk arm will have to wait.

Example:



Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as

= (199-50) + (199-16) = 332

Advantages of SCAN Algorithm

Here are some of the advantages of the SCAN Algorithm.

- High throughput
- Low variance of response time
- Average response time

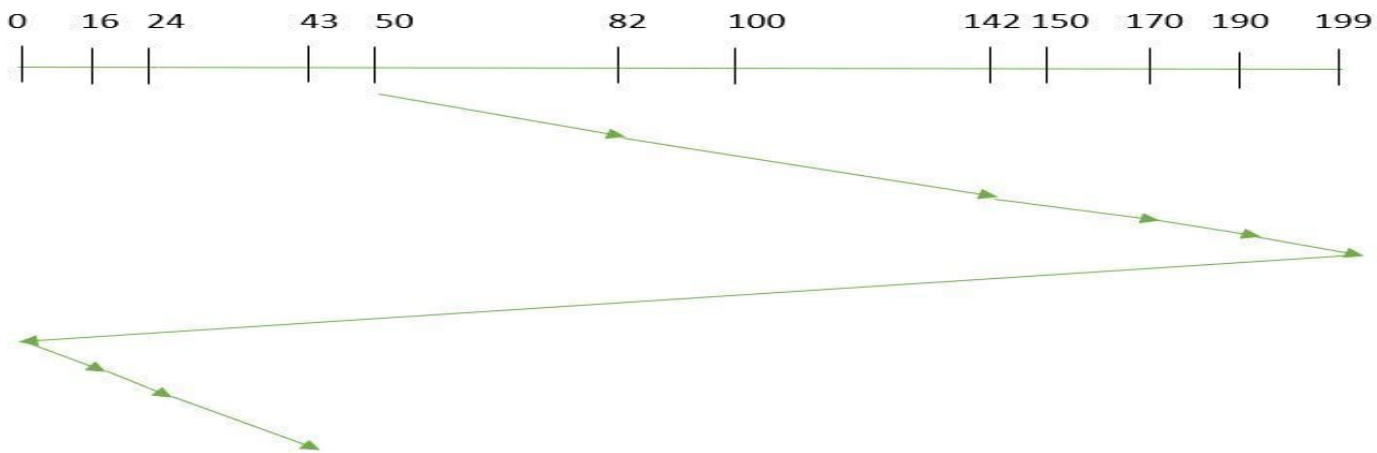
Disadvantages of SCAN Algorithm: Long waiting time for requests for locations just visited by disk arm

4. C-SCAN

In the SCAN algorithm, the disk arm again scans the path that has been scanned, after reversing its direction. So, it may be possible that too many requests are waiting at the other end or there may be zero or few requests pending at the scanned area.

These situations are avoided in the CSCAN algorithm in which the disk arm instead of reversing its direction goes to the other end of the disk and starts servicing the requests from there. So, the disk arm moves in a circular fashion and this algorithm is also similar to the SCAN algorithm hence it is known as C-SCAN (Circular SCAN).

Example:



Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move **"towards the larger value".**

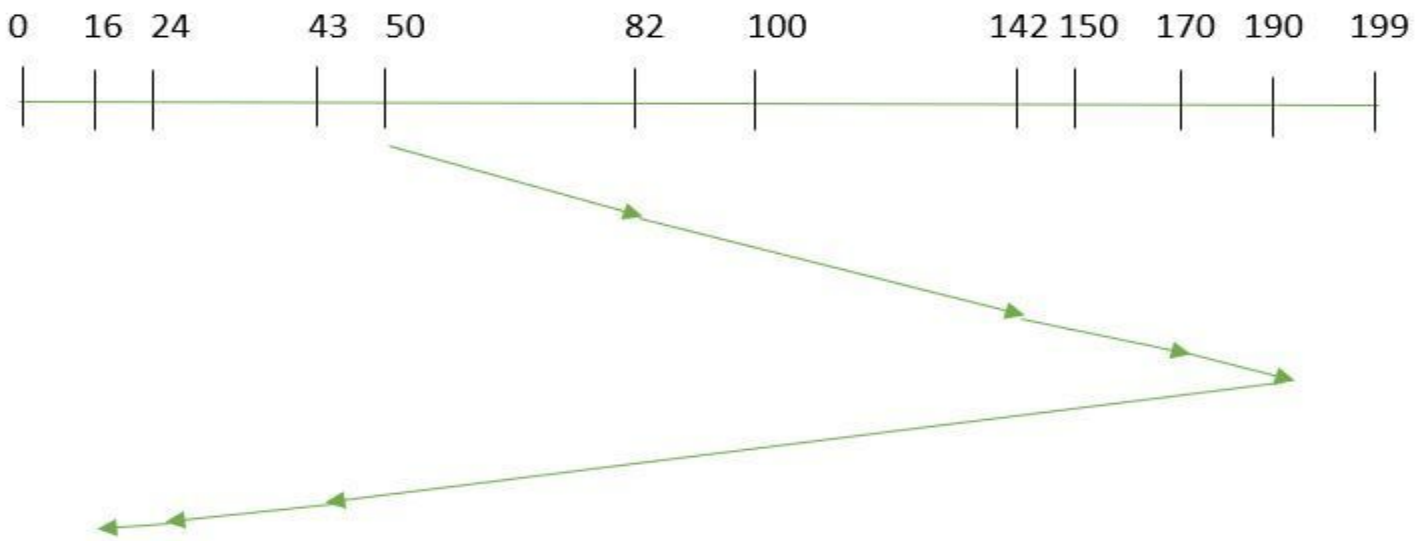
So, the total overhead movement (total distance covered by the disk arm) is calculated as:

=(199-50) + (199-0) + (43-0) = 391

Advantages of C-SCAN Algorithm: Provides more uniform wait time compared to SCAN.

5. LOOK

LOOK Algorithm is similar to the SCAN disk scheduling algorithm except for the difference that the disk arm in spite of going to the end of the disk goes only to the last request to be serviced in front of the head and then reverses its direction from there only. Thus it prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.



Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".

So, the total overhead movement (total distance covered by the disk arm) is calculated as:

= (190-50) + (190-16) = 314

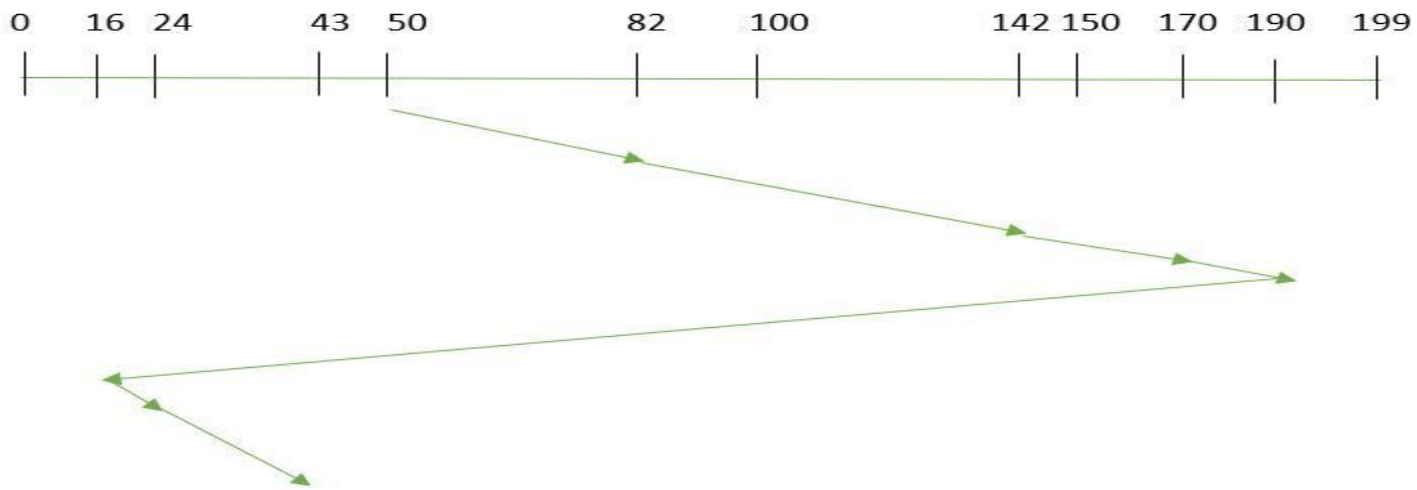
Advantages

- **Reduced Unnecessary Movement:** The disk arm only goes as far as the last request in each direction, avoiding travel to the disk's physical end (unlike SCAN).
- **Faster Response:** Less head movement leads to quicker service for requests.

6. C-LOOK

As LOOK is similar to the SCAN algorithm, in a similar way, C-LOOK is similar to the CSCAN disk scheduling algorithm. In CLOOK, the disk arm in spite of going to the end goes only to the last request to be serviced in front of the head and then from there goes to the other end's last request. Thus, it also prevents the extra delay which occurred due to unnecessary traversal to the end of the disk.

Example: Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value"



So, the total overhead movement (total distance covered by the disk arm) is calculated as

$= (190-50) + (190-16) + (43-16) = 341$

Advantages:

- **Uniform Wait Time:** Requests are serviced in a circular manner, so waiting times are more predictable and fair.
- **Reduced Head Movement**: The arm only goes as far as the last request in one direction, then jumps back, saving time compared to C-SCAN.

Boot Block and Bad Block in Operating System

In a computer system, the **operating system** (OS) is an essential part that performs several functions like disk management, security management, resource management, etc.

Both bad block and boot block are two important features of disk management in an operating system.

- **Boot block** is an important component of an operating system which resides in a region of a hard disk or any other storage device and contains all crucial data and instructions required for initiating the booting process.
- On the other hand, **bad block** is a region or sector of a data storage device which is damaged or malfunctioned and is not reliable to store data.

What are Boot Blocks?

The process of starting or restarting a computer system or any other computing device is called booting. For booting a computer system, a set of data and instructions is required which is managed by the operating system of the computer.

In a computing device, the part of the operating system which resides in a region of a data storage devices like hard disk, optical disk, floppy disk, etc. and contains all the data and instructions required for starting the booting process of the system is referred to as boot block. It is also known as boot sector, as it is a sector in the memory device which contains all the instructions required to boot the system.

Boot block generally resides in the first sector of the data storage device like hard disk and is designed in a standard format so that the BIOS (Basic Input Output System) can understand and execute it.

Components of a Boot Block

The following are the important components of the boot block ?

- **Master Boot Record (MBR)** ? The master boot record (MBR) is a first part of a storage device that contains the boot block, boot code, partition table, and other required data and instructions. In a computer system, the master boot record is an essential part that helps to understand how the storage device is organized and which partition is required to boot.
- **Bootloader** ? Bootloader is a computer program which is responsible for starting the system and loading the operating system into the main memory. It executes all the essential steps required to initiate the booting process.
- **Boot Code** ? Boot code, also called bootstrap code, is another important component of boot block. Bootstrap code consists of all the essential instructions written in low-level language like machine language or assembly language. Bootstrap code performs some important functions, including configuration of system components, initialization of hardware parts, loading of operating system into the main memory, etc.
- **Partition Table** ? Partition table is another important component of the boot block. It is basically a data table that contains information about the different partition of the data storage device. It helps in identifying the active partition on the disk from which the operating system is to be loaded into the main memory.

What are Bad Blocks?

A storage region or sector of a data storage device like hard disk drive, flash drive, optical disk, etc. that cannot be used for storage and retrieval of data due to permanent damage is referred to as a bad block. Sometimes, a **bad block** is also known as a **bad sector.**

In other words, the damaged sector or portion of a storage device is called bad block. In a data storage device, the bad blocks can occur due to various reasons, such as physical damage, software error, wear and tear due to aging, manufacturing defects, etc. In a computer system, the effect of bad blocks could be the loss of data and system instability.

When a bad block occurs in a system and is encountered by the operating system, the operating system takes corrective actions to prevent data loss, system instability and other issues. For this, the operating system is generally empowered with tools to identify and manage the bad blocks. For instance, in Windows operating system, a disk utility named, "Check Disk (chkdsk)" is integrated to scan the file system to detect the bad blocks. Once a bad block on a storage device is detected, the operating system marks it as not for use. Thus, this portion of the disk is not used to store any data.

Types of Bad Blocks

There can be several different types of bad blocks in a storage device.

- **Physical Bad Block** ? The type of bad block that occurred due to any kind of physical damage like wear and tear, magnetic interference, scratches, etc. to the data storage device is known as **physical bad block**. This type of bad block is also referred to as **hard bad block** because it is permanent and cannot be repaired. Therefore, it can result in loss of data permanently.
- **Logical Bad Block** ? Logical bad block is a type of bad block that caused due to software errors like virus infection, file system errors, etc. It is also known as soft bad block because it can be repaired using software tools.
- **Unreadable Bad Block** ? The type of bad block whose data is not available to read due to data corruption or any other issues is called unreadable bad block. Unreadable blocks result in data loss.

Thank You!!